Lab Report

23CSE212 – Principles of Functional Languages

Criteria	Excellent	Good	Poor
Timely Submission			
Correctness of lab assignment			
Total Marks			
Signed By Lab Instructor			

Lab Session No: 4
Date:

Question 1 : What is the type of each of the following expressions?
a. "Haskell" ++ "list"
b. [True, False, True, True]
c. [True, False, 'a']
d. (True, False, 'a')

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
:t "Haskell" ++ "list" :t [True, False, True, True] :t [True, False, 'a'] :t (True, False, 'a')	ghci> :t "Haskell" ++ "list" "Haskell" ++ "list" :: [Char] ghci> :t [True, False, True, True] [True, False, True, True] :: [Bool] ghci> :t [True, False, 'a'] <interactive>:1:15: error: • Couldn't match expected type 'Bool' with actual type 'Char' • In the expression: 'a' In the expression: [True, False, 'a'] ghci> :t (True, False, 'a') (True, False, 'a') :: (Bool, Bool, Char)

Question 2 : Write a script to find the last but one element of a list.
[Input: [1,2,3,4] Output: 3]

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
lastbutone :: [a] -> a lastbutone l = l !! (length(l)-2)	ghci> lastbutone [1..4] 3 ghci> lastbutone [5..10] 9

Question 3 : Write a script to find the kth element of list, where k is the index.
[Input: [1,2,3,4] k=2 Output: 3]
CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
kthelement ::[a] -> Int -> a kthelement l k =l !! k	ghci> kthelement [1..4] 2 3 ghci> kthelement [1..10] 7 8

Question 4 : Write a function IstPalindrome to find out whether a list is a palindrome.
[Input1: [1,2,1] Output: True, Input2="mom" Output2: True]
CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
isPalindrome ::(Eq a) => [a] -> Bool isPalindrome l = l == reverse l	ghci> isPalindrome [1,2,1] True ghci> isPalindrome "mom" True ghci> isPalindrome "hello" False

Question 5 : Define a function duplicate which will duplicate the list 3 times and produce a new list.
Example [1,23] - will give output [1,2,3,1,2,3,1,2,3].
CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
duplicate :: [a] -> [a] duplicate l =take(3*length l) (cycle l)	ghci> duplicate [1,2,3] [1,2,3,1,2,3,1,2,3] ghci> duplicate [1,2,3,5,6,7] [1,2,3,5,6,7,1,2,3,5,6,7,1,2,3,5,6,7]

Question 6 : Split a list by defining a function which takes an input list and an integer n and divides the list into two the first n elements as the first list and the rest as the second list and form a tuple of lists. Let [1 .. 10] be a list and value of n be 4 then the new tuple formed is as ([1,2,3,4], [5,6,7,8,9,10]). Another example I/P splits "amr" 4 & O/P- ["amr",""]

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
split :: [a] -> Int -> ([a],[a]) split l k = splitAt k l	ghci> split [1..10] 4 ([1,2,3,4],[5,6,7,8,9,10]) ghci> split [1..10] 7 ([1,2,3,4,5,6,7],[8,9,10])

Question 7 : Define a function that will slice a list based on the input indices i and k. Consider a list [1 .. 10] and let i = 2 and k = 4 respectively then the resultant list will be [3,4,5].

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
getrange :: [a] -> Int -> Int -> [a] getrange list left right = take (right-left +1) (drop left list)	ghci> getrange [1..10] 2 4 [3,4,5] ghci> getrange [1..20] 2 12 [3,4,5,6,7,8,9,10,11,12,13]

Question 8 : Define a function to remove the kth indexed element from a list. Consider a list [1 .. 10],and value of n be 2, then resultant list will be [1,2,4,5,6,7,8,9,10].

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
removeEle :: [a] -> Int -> [a] removeEle list k = take k list ++ drop (k+1) list	ghci> removeEle [1..10] 5 [1,2,3,4,5,7,8,9,10] ghci> removeEle [1..10] 2 [1,2,4,5,6,7,8,9,10]

Question 9: Define a function that will insert an element n at a particular index i of a list xs. Let xs=[1 .. 10], i=2, n=11, then the output will be [1,2,11,3,4,5,6,7,8,9,10].

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
insert :: [a] -> Int -> a -> [a] insert list index ele = take index list ++ [ele] ++ drop(index) list	ghci> insert [1..10] 2 11 [1,2,11,3,4,5,6,7,8,9,10] ghci> insert [1..20] 0 0 [0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20]

Question 10 : Define a function that takes an integer number n and a value v and creates a list containing n occurrences of v.

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
createlist :: Int -> a ->[a] createlist n v =take n (cycle [v])	ghci> createlist 3 5 [5,5,5] ghci> createlist 5 10 [10,10,10,10,10]

Question 11 : What value has the expression [0, 0.1 .. 1] ? Check your answer and explain any discrepancy there might be between the two.

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
----------	ghci> [0,0.1..1] [0.0,0.1,0.2,0.30000000000000004,0.4, 0.5,0.6000000000000001,0.7000000000000001, 0.8,0.9,1.0]

The expression [0, 0.1 .. 1] in Haskell generates a list of numbers starting from 0, incrementing by 0.1, up to 1. Discrepancies such as 0.30000000000000004 instead of 0.3 and 0.6000000000000001 instead of 0.6 due to floating-point precision errors.

Question 12 : To find all the digits in a string where your function returns True on those characters which are digits:'0',' l'up to' 9'.
CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
checkDigits :: String -> [Bool] checkDigits str = map isDigit str	ghci> checkDigits "Haskell_12" [False,False,False,False,False,False,False,False,True,True] ghci> checkDigits "1234" [True,True,True,True]

Question 13 : Define a function firstLastTup that returns a tuple containing the first and last elements of a list. [firstLastTup [1 .. 6] -- > (1,6)]
CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
firstLastTuple :: [a] -> (a,a) firstLastTuple list = (head list ,last list)	ghci> firstLastTuple [1..6] (1,6) ghci> firstLastTuple [1..16] (1,16)

Question 14 : Define a function firstLast that takes a list and returns a tuple containing the first and last elements of the list. [firstLast [1,2,3,4,5] -- Output: (1,5)]
CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
firstLast :: [a] -> (a,a) firstLast list = (head list ,last list)	ghci> firstLast [1..5] (1,5) ghci> firstLast [5..15] (5,15)

Question 15 : Define a function reverseConcat that takes two lists and concatenates them in reverse order. [reverseConcat [1,2,3] [4,5,6] -- Output: [3,2,1,6,5,4]]

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
reverseConcat ::[a] -> [a] ->[a] reverseConcat list1 list2 =list2 ++ list1	ghci> reverseConcat [1..3] [4..6] [4,5,6,1,2,3] ghci> reverseConcat [6..10] [0..5] [0,1,2,3,4,5,6,7,8,9,10]

Question 16 : Define a function dropTake that takes three arguments: a list, an integer n, and an integer m. It drops the first n elements and takes the next m elements from the list. [dropTake [1,2,3,4,5,6,7,8,9,10] 3 4 -- Output: [4,5,6,7]]

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
dropTake :: [a] -> Int ->Int ->[a] dropTake list n m = take m (drop n list)	ghci> dropTake [1..10] 3 4 [4,5,6,7] ghci> dropTake [1..10] 2 6 [3,4,5,6,7,8]

Question 17 : Define a function inFirstHalf that checks if an element is in the first half of a list. [Sample Input: xs= [1 .. 6], x=3; Sample Output: True]

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
firstHalf ::Eq a => [a] -> a -> Bool firstHalf list k = elem k (take (div (length list) 2) list)	ghci> firstHalf [1..6] 3 True ghci> firstHalf [1..6] 6 False

Question 18 : Define a function trimList that returns a list without its first n and last m elements.(trimList 3 3 [1 .. 10] -- > [4,5,6,7])

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
trimList :: [a] -> Int -> Int ->[a] trimList list n m = take (length list - m-n) (drop n list)	ghci> trimList [1..10] 3 3 [4,5,6,7] ghci> trimList [1..10] 3 5 [4,5]

Question 19. Define a function removeEdges that removes the first and last element of a list. (Sample input: xs=[1 .. 6], output: [2,3,4,5])

CO2: Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
removeEdge :: [a] ->[a] removeEdge list = tail(init list)	ghci> removeEdge [1..6] [2,3,4,5] ghci> removeEdge [5..12] [6,7,8,9,10,11]

Question 20. Define a function sumMiddle that takes a list and returns the sum of all elements except

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
sumMiddle :: Num a => [a] -> a sumMiddle list =sum (tail(init list))	ghci> sumMiddle [1..5] 9 ghci> sumMiddle [1..15] 104